

› Question 1: Loan Eligibility Calculator (Functions)

↳ 2 cells hidden

› Question 2: Function Parameters - Flexible Calculator

↳ 4 cells hidden

✓ Question 3: Nested Functions - Invoice Calculator

Description

Create an invoice calculation system using nested functions.

Requirements

Create a main function `calculate_invoice()` that:

- Accepts base price, tax rate, and optional discount code
- Contains nested functions:
 - `calculate_tax(price, rate)`: Calculates tax amount
 - `apply_discount(price, code)`: Applies discount based on code
 - "DISCOUNT10": 10% off
 - "DISCOUNT20": 20% off
 - "VIP": 25% off
 - `format_currency(amount)`: Formats amount with currency symbol
 - Returns formatted invoice with breakdown

Display itemized invoice

Input/Output Example

```
calculate_invoice(1000, 5, "DISCOUNT10")
```

```
=== Invoice ===
```

```
Base Price: Rs 1,000.00
```

```
Discount (10%): -Rs 100.00
```

```
Subtotal: Rs 900.00
```

```
Tax (5%): Rs 45.00
```

```
Total: Rs 945.00
```

Constraints

- Nested functions should only be accessible within main function
- Validate discount codes
- Handle invalid inputs

```
def calculate_invoice(base_price, tax_rate, discount_code=None):
    if base_price < 0:
        return "Error: Base price cannot be negative."

    if tax_rate < 0:
        return "Error: Tax rate cannot be negative."

    def calculate_tax(price, rate):
        return price * rate / 100

    def apply_discount(price, code):

        discounts = {
            "DISCOUNT10": 10,
            "DISCOUNT20": 20,
            "VIP": 25
        }
        if code is None:
            return price, 0

        if code not in discounts:
            return price, 0

        discount_percent = discounts[code]

        discount_amount = price * discount_percent / 100

        return price - discount_amount, discount_percent

    def format_currency(amount):
        return f"Rs {amount:,.2f}"

    subtotal, discount_percent = apply_discount(
        base_price,
        discount_code
    )
    discount_amount = base_price - subtotal

    tax_amount = calculate_tax(
        subtotal,
        tax_rate
    )

    total = subtotal + tax_amount

    print("=== Invoice ===")
```

```

print(f"Base Price: {format_currency(base_price)}")

if discount_percent > 0:
    print(
        f"Discount ({discount_percent}%): "
        f"-{format_currency(discount_amount)}"
    )
else:
    print("Discount: Rs 0.00")

print(f"Subtotal: {format_currency(subtotal)}")
print(f"Tax ({tax_rate}%): {format_currency(tax_amount)}")
print(f"Total: {format_currency(total)}")

```

```
calculate_invoice(1000, 5, "DISCOUNT10")
```

```

=== Invoice ===
Base Price: Rs 1,000.00
Discount (10%): -Rs 100.00
Subtotal: Rs 900.00
Tax (5%): Rs 45.00
Total: Rs 945.00

```

Question 4: Functions Returning Functions - Currency Converter Factory

Description

Create a currency converter system where a function returns another function.

Requirements

Create a function `currency_converter_factory(rate)` that:

- Accepts exchange rate
- Returns a converter function that converts amounts using that rate

Create multiple converters:

- USD to INR (rate: 83.0)
- EUR to INR (rate: 90.0)
- GBP to INR (rate: 105.0)

Create a function `create_converter_menu()` that returns a dictionary of converter functions

Allow users to convert amounts using selected converter

Input/Output Example

USD to INR Converter created (Rate: 83.0)

EUR to INR Converter created (Rate: 90.0)

GBP to INR Converter created (Rate: 105.0)

Converting \$100 USD to INR: Rs 8,300.00

Converting €50 EUR to INR: Rs 4,500.00

Converting £30 GBP to INR: Rs 3,150.00

Constraints

- Validate exchange rates (must be positive)
- Handle negative amounts appropriately

```
def currency_converter_factory(rate):
    if rate <= 0:
        raise ValueError(
            "Exchange rate must be positive."
        )
    def converter(amount):
        if amount < 0:
            return "Error: Amount cannot be negative."
        return amount * rate
    return converter
def create_converter_menu():
    return {
        "USD": currency_converter_factory(83.0),
        "EUR": currency_converter_factory(90.0),
        "GBP": currency_converter_factory(105.0)
    }
converters = create_converter_menu()
print("USD to INR Converter created (Rate: 83.0)")
print("EUR to INR Converter created (Rate: 90.0)")
print("GBP to INR Converter created (Rate: 105.0)")
usd_amount = 100
eur_amount = 50
gbp_amount = 30

usd_inr = converters["USD"](usd_amount)
eur_inr = converters["EUR"](eur_amount)
gbp_inr = converters["GBP"](gbp_amount)

print(
    f"\nConverting ${usd_amount} USD to INR: "
    f"Rs {usd_inr:,.2f}"
)
```

```
print(
    f"Converting €{eur_amount} EUR to INR: "
    f"Rs {eur_inr:,.2f}"
)

print(
    f"Converting £{gbp_amount} GBP to INR: "
    f"Rs {gbp_inr:,.2f}"
)
```

USD to INR Converter created (Rate: 83.0)
EUR to INR Converter created (Rate: 90.0)
GBP to INR Converter created (Rate: 105.0)

Converting \$100 USD to INR: Rs 8,300.00
Converting €50 EUR to INR: Rs 4,500.00
Converting £30 GBP to INR: Rs 3,150.00

› Question 5: Lambda Functions - Data Processing

↳ 2 cells hidden

› Question 6: Decorators - Function Enhancement

↳ 2 cells hidden

› Question 7: Custom Module Creation

↳ 5 cells hidden

› Question 8: Package Creation and Organization

↳ 10 cells hidden